Algorithm Design and Explanation

Student ID: 24012194

This report explains the logic and idea behind the code I wrote for Task 1.1 to Task 1.4. from the using of lab exercises and other helpful sources. I have used Python for all programs. I also used some external libraries like pandas and DeepFace, and I will mention where I used them.

Task 1.1

It was about calculating degree results for students. We had to take data from two CSV files, one with module information and one with students' marks. For Level 5, we needed to choose the best 100 credits and calculate the weighted average. For Level 6, we used all module marks and found the average. Then, we used the formula (Level6 * 3 + Level5) / 4 to find the final mark. Based on this, we gave degree class like First Class, 2.1, 2.2 and so on. I used pandas to read the CSV files and do the calculations. I used loops and if conditions to check and filter marks. I think pandas made the work easier and faster. I used pandas to save time and reduce code complexity.

Output:

```
PS C:\Users\inder singh\Downloads\task1_4>  c:; cd 'c:\Users\inder singh\Downloads\task1_4'; & 'c:\Users\inder singh\deeptest\Scripts\python.exe' 'c:\Users\inder
 singh\.vscode\extensions\ms-python.debugpy-2025.6.0-win32-x64\bundled\libs\debugpy\launcher' '56221' '--' 'C:\Users\inder singh\Downloads\task1_4\task1_1\task1_
1_degree_calc.py'
Current directory: C:\Users\inder singh\Downloads\task1_4
Files in directory: ['task1_1', 'task1_2', 'task1_3', 'task1_4']
C:\Users\inder singh\Downloads\task1_4\task1_1\task1_1_degree_calc.py:102: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This beha
vior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include_groups=False` to exclude the
 groupings or explicitly select the grouping columns after groupby to silence this warning.
  final_results_df = merged_df.groupby('StudentID').apply(calculate_degree_with_pass_check).reset_index(drop=True)
✅ Degree results saved to 'degree_results_with_pass_check.csv'
✅ Script ran successfully.
PS C:\Users\inder singh\Downloads\task1_4>
```

Task 1.2

I wrote a program to calculate total password combinations. Each password starts with 3 lowercase letters and the rest are numbers. For example, if length is 4, then it will be 3 letters and 1 number. So total combinations = 26 * 26 * 26 * 10. I used simple math and loops to get the answer. For length 5 and 6 also, I just increased the number part. I used print statements to show the final count. This task was small but helped me learn about how to use loops and multiplication logic to solve such problems. I used basic math and loops because it was simple and didn't need any complex code

Output:

```
PS C:\Users\inder singh\Downloads\task1_4> ^C
PS C:\Users\inder singh\Downloads\task1_4>
PS C:\Users\inder singh\Downloads\task1_4>  c:; cd 'c:\Users\inder singh\Downloads\task1_4'; & 'c:\Users\inder singh\deeptest\Scripts\python.exe' 'c:\Users\inder
 singh\.vscode\extensions\ms-python.debugpy-2025.6.0-win32-x64\bundled\libs\debugpy\launcher' '56270' '--' 'C:\Users\inder singh\Downloads\task1_4\task1_2\task1_
2_password_generator.py'
Enter the password length (try 4, 5 or 6): 4
Total valid passwords: 4500
All saved to valid_passwords.txt
PS C:\Users\inder singh\Downloads\task1_4>
Ln 1, Col 1   Spaces: 4   UTF-8   LF   {} Python   3.11.7 ('deeptes
```

Task 1.3

It was more advanced. It was a route-finding program using a dataset of UK railway stations. Each route had a cost and time. I used Dijkstra's Algorithm to find the shortest route, either by cost or by time. I used dictionaries to store the graph, and a priority queue to select the next best station. The user can type the start and end station and choose whether they want the fastest or cheapest route. The program then gives the total cost/time and the station path. This was a little complex, but I understood that graph and algorithm help in solving such real-life tasks like Google Maps. Using a priority queue helped make the route finding faster and used less memory compared to brute force. I used dictionaries and a priority queue because they are efficient and help make the search faster.

Output:

```
PS C:\Users\inder singh\Downloads\task1_4>
PS C:\Users\inder singh\Downloads\task1_4>  c:; cd 'c:\Users\inder singh\Downloads\task1_4'; & 'c:\Users\inder singh\deeptest\Scripts\python.exe' 'c:\Users\inder
 singh\.vscode\extensions\ms-python.debugpy-2025.6.0-win32-x64\bundled\libs\debugpy\launcher' '57520' '--' 'C:\Users\inder singh\Downloads\task1_4\task1_3\task1_
3_train_search.py'
Enter starting station: bristol temple meads
Enter destination station: london
Do you want the 'cheapest' or 'fastest' route? cheapest
The total cost is 152 with the following route: ['bristol temple meads', 'newport', 'bristol parkway', 'cheltenham spa', 'worcester', 'evesham', 'moreton-in-mars
h', 'charlbury', 'oxford', 'banbury', 'bicester', 'princes risborough', 'high wycombe', 'london']
PS C:\Users\inder singh\Downloads\task1_4> ^C
Ln 13, Col 1   Spaces: 4   UTF-8   CRLF   {} Python   3.11.7 ('deeptest')
```

Task 1.4

It was about face recognition. In this, I had to compare one known image to a group of images and find the matching one. I tried using the face recognition library first, but it gave errors due to image type and dlib issues. Then I used DeepFace which was better and easy. I made two versions: one serial where we compare images one by one, and one parallel where we use multithreading to check faster. In the end, the program prints which image matched with the known face. I learned how face matching works and how AI can be used in programs. Also, I had to fix many errors during setup, which helped me understand about environment and packages. I used DeepFace to avoid dlib issues and save time with less setup problems.

Output:

```
✅ Compared with f72f062a7fb0a250.jpg → Match: False
✅ Compared with f79b3b10baf65396.jpg → Match: False
✅ Compared with f81a3d308a5caf27.jpg → Match: False
✅ Compared with f83e97271cc645ff.jpg → Match: False
✅ Compared with f90ed1de38535266.jpg → Match: False
✅ Compared with known_man_rgb.jpg → Match: True
⏱Time taken: 244.90 seconds
PS C:\Users\inder singh\Downloads\task1_4>
```

More Explanation About Tasks:

For design

I mostly used dictionaries, loops, and pandas dataframes. For Task 1.3 and 1.4, I used standard algorithms like Dijkstra and face detection through libraries, and I made sure to learn how they work. I applied them correctly in my code and focused on making the overall program simple and easy to understand. I referred to lab exercises and online documentation but wrote my own final code.

I faced issues like file not found, library not installing, and image format problems. I searched online, watched videos and asked for help to fix them. I also tested the code with different inputs to make sure it works correctly.

Before writing the code:

I also created small flowcharts and thought about how the data should flow. This helped me keep the logic simple and avoid mistakes. I also made sure my code was efficient, especially in Task 1.3 where I used a priority queue to keep memory usage low and make route-finding faster. In Task 1.4, using DeepFace was a good choice because it reduced errors compared to manual setup with dlib. For every task, I tried to test with different inputs to make sure the program gives correct results every time.

In short

This coursework helped me learn how to plan, write and fix Python programs using logic and external libraries. It also showed how algorithms like Dijkstra and AI-based face recognition are used in real life. I learned a lot and I feel more confident in writing programs now.

Libraries used:

- **Pandas (for data handling)**

- **face_recognition (initially tried)**
- **deepface (used for final face recognition)**
- **threading (for parallel code)**